Autômatos de pilha

O que o ANTLR faz com a gramática de vocês

Encontro 11 | 7 de outubro

Linguagens Formais e Compiladores

1

O que acontece por dentro

Cada regra vira uma função

```
expr : termo (('+'|'-') termo)* ;
termo : fator (('*'|'/') fator)* ;
fator : NUM | '(' expr ')' ;
```

Cada regra vira uma função

```
expr : termo (('+'|'-') termo)* ;
termo : fator (('*'|'/') fator)* ;
fator : NUM | '(' expr ')' ;
```



Cada regra vira uma função

```
expr : termo (('+'|'-') termo)* ;
termo : fator (('*'|'/') fator)* ;
fator : NUM | '(' expr ')' ;
```



Uma regra chamando outra vira uma função chamando outra.

E chamada de função é uma pilha

Quando `expr()` chama `termo()`, o computador guarda onde parou no `expr()` para voltar depois.

E chamada de função é uma pilha

Quando `expr()` chama `termo()`, o computador guarda onde parou no `expr()` para voltar depois.

Essa é a pilha de chamadas. E ela é a pilha do autômato.

E chamada de função é uma pilha

Quando `expr()` chama `termo()`, o computador guarda onde parou no `expr()` para voltar depois.

Essa é a pilha de chamadas. E ela é a pilha do autômato.

Vocês não precisam programar a pilha. Ela já existe na linguagem.

Vendo a pilha crescer

Analisando `2 + 3 * ( 4 - 1 )`

a entrada

2	+	3	*	(	4	-	1	)

lido / lendo / falta

a gramática

```
expr  :   termo (('+'|'-') termo)*
termo :   fator (('*'|'/') fator)*
fator :   NUM | '(' expr ')'
```

a pilha

expr ← agora

Toda análise começa pela regra inicial: expr.

Vendo a pilha crescer

Analisando `2 + 3 * ( 4 - 1 )`

a entrada

2	+	3	*	(	4	-	1	)

lido / lendo / falta

a gramática

```
expr  :  termo (('+'|'-') termo)*
termo :  fator (('*'|'/') fator)*
fator :  NUM | '(' expr ')'
```

a pilha

expr ← agora

Para fazer um expr, a regra manda fazer um termo primeiro.

Vendo a pilha crescer

Analisando `2 + 3 * ( 4 - 1 )`

a entrada

2	+	3	*	(	4	-	1	)

lido / lendo / falta

a gramática

```
expr  :   termo (('+'|'-') termo)*
termo :   fator (('*'|'/') fator)*
fator :   NUM | '(' expr ')'
```

a pilha



Para fazer um termo, a regra manda fazer um fator primeiro.

Vendo a pilha crescer

Analisando `2 + 3 * ( 4 - 1 )`

a entrada

2	+	3	*	(	4	-	1	)

lido / lendo / falta

a gramática

```
expr  :   termo (('+'|'-') termo)*
termo :   fator (('*'|'/') fator)*
fator :   NUM | '(' expr ')'
```

a pilha



fator aceita NUM. O 2 serve: consome e volta.

Vendo a pilha crescer

Analisando `2 + 3 * ( 4 - 1 )`

a entrada

2	+	3	*	(	4	-	1	)

lido / lendo / falta

a gramática

```
expr  :   termo (('+'|'-') termo)*
termo :   fator (('*'|'/') fator)*
fator :   NUM | '(' expr ')'
```

a pilha



De volta no termo. Depois do fator ele quer * ou /. Veio +, então acaba.

Vendo a pilha crescer

Analisando `2 + 3 * ( 4 - 1 )`

a entrada

2	+	3	*	(	4	-	1	)

lido / lendo / falta

a gramática

```
expr  :  termo (('+'|'-') termo)*
termo :  fator (('*'|'/') fator)*
fator :  NUM | '(' expr ')'
```

a pilha



De volta no expr. Ele aceita +, então consome e pede outro termo.

Vendo a pilha crescer

Analisando `2 + 3 * ( 4 - 1 )`

a entrada

2	+	3	*	(	4	-	1	)	lido / lendo / falta

a gramática

```
expr   :   termo (('+'|'-') termo)*
termo  :   fator (('*'|'/') fator)*
fator  :   NUM | '(' expr ')'
```

a pilha



Descemos de novo até fator. Agora o símbolo é (: a outra alternativa.

Vendo a pilha crescer

Analisando `2 + 3 * ( 4 - 1 )`

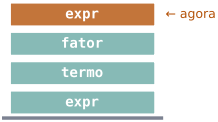
a entrada



a gramática

```
expr  :  termo (('+'|'-') termo)*
termo :   fator (('*'|'/') fator)*
fator :   NUM | '(' expr ')'
```

a pilha



E essa alternativa chama expr DE NOVO. A regra se usa dentro de si mesma.

Vendo a pilha crescer

Analisando `2 + 3 * ( 4 - 1 )`

a entrada

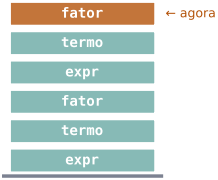
2	+	3	*	(	4	-	1	)

lido / lendo / falta

a gramática

```
expr  :  termo (('+'|'-') termo)*
termo :  fator (('*'|'/') fator)*
fator :  NUM | '(' expr ')'
```

a pilha



Seis níveis abertos. Cada um é uma regra esperando o resto chegar.

O que a animação mostrou

`expr` chama `termo`	porque a regra de `expr` começa com `termo`
`termo` chama `fator`	porque a regra de `termo` começa com `fator`
`fator` chama `expr`	porque uma alternativa dele é `'(' expr ')'`

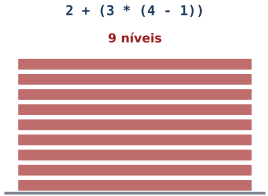
O que a animação mostrou

`expr` chama `termo`	porque a regra de `expr` começa com `termo`
`termo` chama `fator`	porque a regra de `termo` começa com `fator`
`fator` chama `expr`	porque uma alternativa dele é `'('` `expr` `')'`

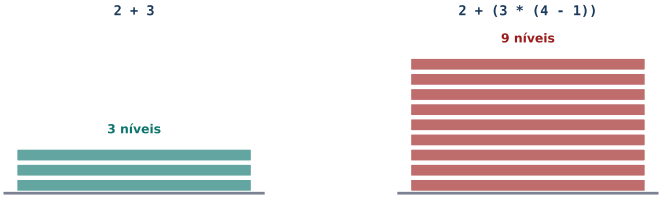
A pilha é só o reflexo de quem a gramática mandou chamar.

A profundidade é o aninhamento



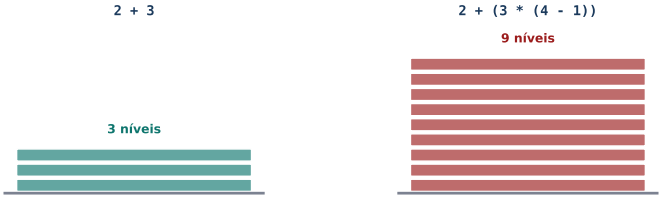


A profundidade é o aninhamento



Cada nível é uma regra que começou e ainda não terminou.

A profundidade é o aninhamento



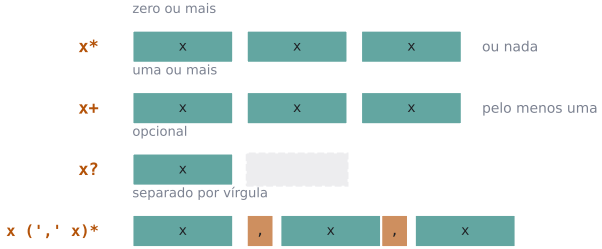
Cada nível é uma regra que começou e ainda não terminou.

E a profundidade não tem limite. Por isso autômato finito não dá.

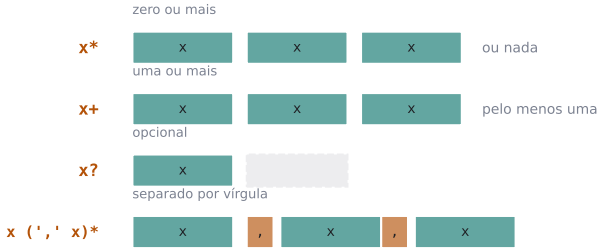
2

Padrões que vocês vão precisar

Os quatro que resolvem quase tudo



Os quatro que resolvem quase tudo



Agora um por um, com a sintaxe do ANTLR.

Uma lista de coisas

```
programa : instrucao* EOF ; // zero ou mais
bloco : '{' instrucao+ '}' ; // uma ou mais
```

Uma lista de coisas

```
programa : instrucao* EOF ; // zero ou mais
bloco : '{' instrucao+ '}' ; // uma ou mais
```

- `*` pode não ter nenhuma
- `+` precisa ter pelo menos uma

Uma lista de coisas

```
programa : instrucao* EOF ; // zero ou mais
bloco : '{' instrucao+ '}' ; // uma ou mais
```

- `*` pode não ter nenhuma
- `+` precisa ter pelo menos uma

Programa vazio é válido? Se sim, use `*`.

Uma coisa opcional

```
se : 'se' '(' expr ')' bloco ('senao' bloco)? ;
```

Uma coisa opcional

```
se : 'se' '(' expr ')' bloco ('senao' bloco)? ;
```

O ? diz que o senao pode vir ou não.

Uma coisa opcional

```
se : 'se' '(' expr ')' bloco ('senao' bloco)? ;
```

O ? diz que o senao pode vir ou não.

Cuidado: isso é o "senão pendente".

Uma coisa opcional

```
se : 'se' '(' expr ')' bloco ('senao' bloco)? ;
```

O ? diz que o senao pode vir ou não.

Cuidado: isso é o "senão pendente".

Com dois se e um senao, a qual deles ele pertence?
O ANTLR resolve ligando ao **mais próximo**, que é o que todo mundo espera.

Separado por vírgula

```
args : expr (',' expr)* ; // um ou mais
args : (expr (',' expr)*)? ; // zero ou mais
```

Separado por vírgula

```
args : expr (',' expr)* ; // um ou mais
args : (expr (',' expr)*)? ; // zero ou mais
```

O primeiro item vem sozinho. Os seguintes vêm com a vírgula junto.

Separado por vírgula

```
args : expr (',' expr)* ; // um ou mais
args : (expr (',' expr)*)? ; // zero ou mais
```

O primeiro item vem sozinho. Os seguintes vêm com a vírgula junto.

A segunda forma permite lista vazia, para `f()` sem argumentos.

Precedência: a ordem decide

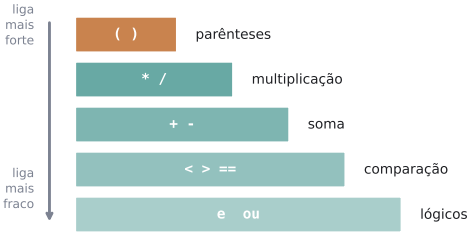
```
expr : expr ('*'|'/') expr // mais forte: vem antes
     | expr ('+'|'-') expr
     | expr ('<'|'>'|'==') expr
     | expr 'e' expr
     | '(' expr ')'
     | NUMERO | IDENT
     ;
```

Precedência: a ordem decide

```
expr : expr ('*'|'/') expr // mais forte: vem antes
     | expr ('+'|'-') expr
     | expr ('<'|'>'|'==') expr
     | expr 'e' expr
     | '(' expr ')'
     | NUMERO | IDENT
     ;
```

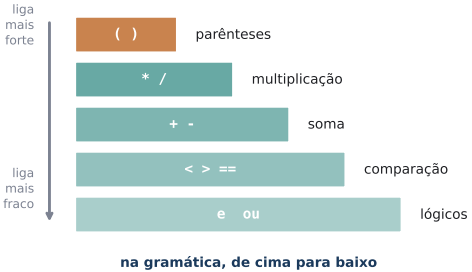
Trocar duas linhas de lugar muda o resultado dos programas, e não dá erro.

A escada



na gramática, de cima para baixo

A escada



Quem está no topo é avaliado primeiro, e aparece primeiro na gramática.

3
Quando der errado

Primeiro: o que é recursão à esquerda

```
expr : expr '+' termo ;
```

Primeiro: o que é recursão à esquerda

```
expr : expr '+' termo ;
```

A regra `expr` começa chamando `expr`, **antes de consumir qualquer símbolo**.

Primeiro: o que é recursão à esquerda

```
expr : expr '+' termo ;
```

A regra `expr` começa chamando `expr`, **antes de consumir qualquer símbolo**.

Compare com o que vimos: lá, `expr` começava com `termo`.

Por que isso é perigoso

Para fazer um `expr`, faça um `expr`. Para fazer esse `expr`, faça um `expr`...

```
expr
expr > expr
expr > expr > expr
expr > expr > expr > expr
```

...sem nunca ler um símbolo

Por que isso é perigoso

Para fazer um `expr`, faça um `expr`. Para fazer esse `expr`, faça um `expr`...

```
expr
expr > expr
expr > expr > expr
expr > expr > expr > expr
```
...sem nunca ler um símbolo

A pilha cresce para sempre. O programa trava.

Mas o ANTLR 4 resolve isso sozinho

Ele reescreve a regra internamente, de um jeito que não entra em laço.

Mas o ANTLR 4 resolve isso sozinho

Ele reescreve a regra internamente, de um jeito que não entra em laço.

Então vocês *podem* escrever `expr : expr '+' expr` e funciona.

Mas o ANTLR 4 resolve isso sozinho

Ele reescreve a regra internamente, de um jeito que não entra em laço.

Então vocês *podem* escrever `expr : expr '+' expr` e funciona.

Ferramentas mais antigas não faziam isso, e exigiam reescrever à mão.
Se vocês virem isso em livros, é esse o motivo.

Então quando aparece o erro?

```
error: rule EXPR is left-recursive
```

Então quando aparece o erro?

```
error: rule EXPR is left-recursive
```

Repare na **MAIÚSCULA**. Vocês escreveram uma regra de *lexer*.

Então quando aparece o erro?

```
error: rule EXPR is left-recursive
```

Repare na **MAIÚSCULA**. Vocês escreveram uma regra de *lexer*.

```
EXPR : EXPR '+' EXPR ;
```
lexer: **recusa**

```
expr :  expr '+' expr ;
```
parser: **aceita**

Então quando aparece o erro?

```
error: rule EXPR is left-recursive
```

Repare na **MAIÚSCULA**. Vocês escreveram uma regra de *lexer*.

```
EXPR : EXPR '+' EXPR ;
```
lexer: **recusa**

```
expr :  expr '+' expr ;
```
parser: **aceita**

A correção é trocar a caixa da letra.

“mismatched input”

```
line 3:12 mismatched input ';' expecting ')'
```

“mismatched input”

```
line 3:12 mismatched input ';' expecting ')'
```

`line 3:12`	linha 3, coluna 12
`mismatched input ';'`	foi isso que o parser encontrou
`expecting ')'`	e era isso que ele queria

“mismatched input”

```
line 3:12 mismatched input ';' expecting ')'
```

`line 3:12`	linha 3, coluna 12
`mismatched input ';'`	foi isso que o parser encontrou
`expecting ')'`	e era isso que ele queria

O *expecting* é a parte útil: ele diz em que ponto da regra o parser estava quando se perdeu.

Causa mais comum do mismatched

A gramática não previu aquela forma. Quase sempre é um dos dois:

Causa mais comum do mismatched

A gramática não previu aquela forma. Quase sempre é um dos dois:

esqueceu uma alternativa	a regra não aceita aquele caso
esqueceu um símbolo	faltou o `';'` ou o `')'` na regra

Causa mais comum do mismatched

A gramática não previu aquela forma. Quase sempre é um dos dois:

esqueceu uma alternativa	a regra não aceita aquele caso
esqueceu um símbolo	faltou o `';'` ou o `')'` na regra

Leia o *expecting*, vá na regra que o contém,
e veja o que mais poderia aparecer ali.

“extraneous input”

```
line 5:0 extraneous input 'imprime' expecting <EOF>
```

“extraneous input”

```
line 5:0 extraneous input 'imprime' expecting <EOF>
```

O parser achou que o programa tinha **acabado**, e sobrou texto.

“extraneous input”

```
line 5:0 extraneous input 'imprime' expecting <EOF>
```

O parser achou que o programa tinha **acabado**, e sobrou texto.

Significa que a regra inicial casou com só uma parte do arquivo.

“extraneous input”

```
line 5:0 extraneous input 'imprime' expecting <EOF>
```

O parser achou que o programa tinha **acabado**, e sobrou texto.

Significa que a regra inicial casou com só uma parte do arquivo.

Causa comum: a regra aceita uma instrução só, quando deveria aceitar várias.
Faltou o $*$ ou o $+$.

E o erro que não dá erro

Se você esquecer o `EOF` na regra inicial:

```
programa : instrucao* ; // sem EOF
```

E o erro que não dá erro

Se você esquecer o `EOF` na regra inicial:

```
programa : instrucao* ; // sem EOF
```

O parser lê o que entende, para, e *não reclama* do resto.

E o erro que não dá erro

Se você esquecer o EOF na regra inicial:

```
programa : instrucao* ; // sem EOF
```

O parser lê o que entende, para, e *não reclama* do resto.

Metade do arquivo é ignorada em silêncio, e tudo parece funcionar.

E o erro que não dá erro

Se você esquecer o `EOF` na regra inicial:

```
programa : instrucao* ; // sem EOF
```

O parser lê o que entende, para, e *não reclama* do resto.

Metade do arquivo é ignorada em silêncio, e tudo parece funcionar.

Ponha o `EOF`. Sempre.

Como depurar a gramática

1. teste uma regra por vez, assim que escrever
2. comece pela menor: `expr` antes de `programa`
3. use `antlr4-parse Sua.g4 nomeDaRegra -tree`

Como depurar a gramática

1. teste uma regra por vez, assim que escrever
2. comece pela menor: `expr` antes de `programa`
3. use `antlr4-parse Sua.g4 nomeDaRegra -tree`

O terceiro parâmetro é **qualquer** regra, não só a inicial.
Dá para testar só a expressão, sem o programa inteiro.

Como depurar a gramática

1. teste uma regra por vez, assim que escrever
2. comece pela menor: `expr` antes de `programa`
3. use `antlr4-parse Sua.g4 nomeDaRegra -tree`

O terceiro parâmetro é **qualquer** regra, não só a inicial.
Dá para testar só a expressão, sem o programa inteiro.

Escrever tudo e testar no fim é a receita para não achar o erro.

O que fica

- Cada regra de parser vira uma função, e regra chamando regra vira função chamando função.

O que fica

- Cada regra de parser vira uma função, e regra chamando regra vira função chamando função.
- A pilha de chamadas é a pilha do autômato: vocês não precisam programá-la.

O que fica

- Cada regra de parser vira uma função, e regra chamando regra vira função chamando função.
- A pilha de chamadas é a pilha do autômato: vocês não precisam programá-la.
- A profundidade da pilha é o aninhamento da entrada.

O que fica

- Cada regra de parser vira uma função, e regra chamando regra vira função chamando função.
- A pilha de chamadas é a pilha do autômato: vocês não precisam programá-la.
- A profundidade da pilha é o aninhamento da entrada.
- `*` e `+` para listas, `?` para opcional, `(',' x)*` para separado por vírgula.

O que fica

- Cada regra de parser vira uma função, e regra chamando regra vira função chamando função.
- A pilha de chamadas é a pilha do autômato: vocês não precisam programá-la.
- A profundidade da pilha é o aninhamento da entrada.
- `*` e `+` para listas, `?` para opcional, `(',' x)*` para separado por vírgula.
- A ordem das alternativas define a precedência, e errar não dá erro.

O que fica

- Cada regra de parser vira uma função, e regra chamando regra vira função chamando função.
- A pilha de chamadas é a pilha do autômato: vocês não precisam programá-la.
- A profundidade da pilha é o aninhamento da entrada.
- `*` e `+` para listas, `?` para opcional, `(',' x)*` para separado por vírgula.
- A ordem das alternativas define a precedência, e errar não dá erro.
- Leia o *expecting* nas mensagens, e não esqueça o `EOF`.

O que fica

- Cada regra de parser vira uma função, e regra chamando regra vira função chamando função.
- A pilha de chamadas é a pilha do autômato: vocês não precisam programá-la.
- A profundidade da pilha é o aninhamento da entrada.
- `*` e `+` para listas, `?` para opcional, `(',' x)*` para separado por vírgula.
- A ordem das alternativas define a precedência, e errar não dá erro.
- Leia o *expecting* nas mensagens, e não esqueça o `EOF`.

Agora: continuem a gramática de vocês.